Практическая работа 5 – Теоретический конспект
Тема: Моделирование архитектуры «клиент–сервер»

Аннотация

В работе систематизированы теоретические положения о клиент–серверной архитектуре программных систем. Показаны принципы разделения ролей клиента и сервера, типовые модели взаимодействия (синхронные/асинхронные), договорные API-интерфейсы, а также аспекты надёжности, производительности, наблюдаемости, безопасности и развертывания.

Содержание

1. Введение
2. Нормативные ссылки
3. Термины, определения, сокращения
4. Архитектурные основы «клиент–сервер»
5. Модели взаимодействия и контракт API
6. Надёжность и устойчивость взаимодействий
7. Производительность и эффективность
8. Наблюдаемость (логирование, метрики, трассировка)
9. Безопасность
10. Развёртывание и топологии
11. Диаграммы и нотации для ПР-5
12. Заключение

1 Введение

Клиент–серверная архитектура — базовая организация распределённых приложений, при которой функциональность разделяется между инициатором запроса (клиентом) и поставщиком вычислительных/данных услуг (сервером). В современных ИС она реализуется поверх стандартных протоколов прикладного уровня (HTTP/1.1, HTTP/2, HTTP/3), поверх надёжного транспорта (обычно TCP), либо с использованием брокеров сообщений.

Пятая практическая работа посвящена **моделированию**: студент должен уметь оформить поведение, контракты, границы отказов и критерии качества **диаграммами и спецификациями**, пригодными для коммуникации с заказчиком и командой разработки.

2 Нормативные ссылки

В настоящем материале использованы и упоминаются следующие стандарты и источники (для ориентира при оформлении артефактов):

- ГОСТ 7.32-2017 «Отчёт о НИР. Структура и правила оформления».
- ISO/IEC 25010:2011 «Systems and software engineering — System and software quality models».
- OMG UML 2.x, BPMN 2.0; C4 Model (S. Brown).
- OpenAPI Specification 3.0/3.1; AsyncAPI 2.x.

- RFC 9110/9111 (HTTP Semantics/Caching), RFC 9112 (HTTP/1.1), RFC 9114 (HTTP/3), RFC 7540 (HTTP/2).
- R. Fielding «Architectural Styles and the Design of Network-based Software Architectures» (REST).

3 Термины, определения, сокращения

Клиент — инициатор взаимодействия, формирует запросы и интерпретирует ответы.

Сервер — поставщик сервиса/данных, принимает запросы, возвращает ответы.
API — программный интерфейс приложения, договор о структуре и семантике обмена.

SLA/SLO/SLI — соглашение/цель/индикатор качества сервиса.
Идемпотентность — свойство операции давать одинаковый результат при повторе.

Ретраи (повторы) — повторные попытки выполнения операции при сбое.

Circuit Breaker — шаблон выключателя для предотвращения каскадных отказов.

Backoff (джиттер) — стратегия увеличения интервала между повторами.

Keep-Alive — удержание TCP/HTTP-соединения для повторных запросов.

CQS/CQRS — разделение команд и запросов/разделение чтения и записи.

4 Архитектурные основы «клиент–сервер»

4.1 Роли и ответственность

Клиент отвечает за формирование запроса, валидацию вводимых данных на UI-уровне, повторение безопасных операций и корректное завершение соединений. Сервер отвечает за идемпотентность, транзакционность, целостность данных, авторизацию и аудит.

4.2 Варианты организации

- **Толстый/тонкий клиент** (логика на стороне клиента против «тонкого» UI).
- **2-tier/3-tier** (клиент–сервер; клиент–сервер–БД/брокер).
- **Стейтлес/стейтфул** — предпочтителен стейтлес на уровне API; состояние сессии — в токене или хранилище.
- **Синхронные/асинхронные** цепочки: запрос-ответ vs обмен событиями.

4.3 Коммуникационные стили

- **REST-подобное HTTP API** (ресурсы, методы, коды состояния).
- **RPC (gRPC/JSON-RPC)** — акцент на операциях/контрактах.
- **Событийная интеграция** — брокер сообщений, очереди, «pub/sub».

---

5 Модели взаимодействия и контракт API

5.1 Семантика HTTP

Методы **GET** (безопасный, идемпотентный), **PUT** (идемпотентный), **POST** (неидемпотентный), **DELETE** (идемпотентный по договору), коды 2xx/4xx/5xx,

заголовки Cache-Control, ETag, If-None-Match, Authorization, пользовательские X-Request-ID, Idempotency-Key.

5.2 Проектирование контрактов

- **Ресурсы и маршруты**: стабильные URI, версионирование (/v1/...).
- **Схемы данных**: JSON-schema/OpenAPI компоненты, обязательность полей, инварианты.
- **Ошибки**: единый формат (код, сообщение, детали, корреляционный id).
- **Идемпотентность POST**: ключ идемпотентности + уникальные ограничения на стороне сервера.
- **Пагинация и фильтрация**: курсоры/offset, лимиты.

5.3 Контракт-первый (spec-first)

Спецификация (OpenAPI/AsyncAPI) выступает источником правды. Реализация и тесты отталкиваются от неё. В рамках ПР-5 допускается **только спецификация** и диаграммы (без написания кода).

6 Надёжность и устойчивость взаимодействий

6.1 Таймауты и ретраи

Таймауты должны быть конечными и согласованными; повторы — **с экспоненциальным backoff и джиттером**. Автоматические ретраи по умолчанию — для безопасных чтений (**GET**).

6.2 Идемпотентность

Для изменяющих операций обеспечивается идемпотентность:

- на уровне контракта — ключ Idempotency-Key/детерминированные ключи запроса;
- на уровне данных — уникальные индексы/ограничения, «ведомости» выполненных операций;
- на уровне процессов — саги/компенсации.

6.3 Деградация и изоляция

Circuit breaker, bulkhead (разделение пулов), rate limiting, ручки деградации (возврат кэшированных/упрощённых ответов).

7 Производительность и эффективность

Бюджет задержек (p95/p99), удержание соединений (keep-alive), коалесценция запросов, HTTP/2 мультиплексирование, HTTP/3 (QUIC). Кэширование на клиентах и прокси (Cache-Control/ETag), сжатие полезной нагрузки, потоковая передача (streaming) для длинных ответов. Для «тяжёлых» операций — асинхронные очереди и отложенная обработка с уведомлением клиента.

8 Наблюдаемость

Логи: структурированный формат, уровни, корреляционный X-Request-ID.

Метрики: золотые сигналы (latency, traffic, errors, saturation), бизнес-метрики.

Трейсинг: распределённые трассы (trace/span id, контекст); визуализация критических путей.

SLO/SLI: формулирование и привязка к точкам измерения (по связям/операциям контракта).

9 Безопасность

Аутентификация (например, токены/OAuth2/OIDC), авторизация (RBAC/ABAC), шифрование канала (TLS), защита от повторов/подмены, ограничение скоростей, валидация и нормализация входов, аудит действий. Разделение секретов (хранилища секретов), принцип наименьших привилегий для сервисных аккаунтов.

10 Развёртывание и топологии

Базовая топология: клиент(ы) ↔ балансировщик ↔ сервер(ы) ↔ БД/кэш/брокер.

Границы отказов: отдельные зоны/узлы, репликации, автофейловер.

Масштабирование: горизонтальное на уровне стейтлес-слоёв; sticky-session — только при необходимости.

Выпуски: blue-green/canary, feature-flags.

Документация: диаграмма Deployment/C4 Containers с подписью SLO/ограничений на каждую связь.

11 Диаграммы и нотации для ПР-5 (без кода)

Рекомендуется выбрать 2–3 нотации, которые совместно покрывают поведение, контракт и развёртывание. Возможные связки:

1. **BPMN 2.0 Collaboration** (Client/Server как пулы + message flows)
   — показать основной и аварийные потоки (таймер/ошибка/компенсация).
2. **C4 Dynamic Diagram** (поведение поверх контейнерной модели)
   — пронумерованные шаги взаимодействий с пометками SLO/таймаутов.
3. **UML Sequence** (диаграмма последовательностей)
   — нормальный ход + ветки alt: таймаут, повтор, отказ.
4. **UML State Machine** для жизненного цикла запроса/ресурса
   — состояния: Draft → Pending → Succeeded/Failed/Compensated; переходы с кодами ответов.
5. **Deployment / C4 Containers**
   — узлы, соединения, границы отказов, места масштабирования, артефакты (БД/очереди).
6. **Service Blueprint** (по желанию)
   — фронт-/бэк-стейдж: пользовательские шаги ↔ конкретные вызовы API.

11.1 Правила оформления диаграмм

- Единая легенда (обозначения таймаутов, ретраев, SLO/SLI).
- Согласованная нотация шрифтов/цвета (светлая тема, высокий контраст, читаемо с 5–7 м).

- Под диаграммой — «карточка» с предпосылками и инвариантами (что всегда истинно).
- Версионирование диаграмм как кода (PlantUML/Mermaid/Structurizr DSL) — хранить в VCS.
- Для каждой схемы — связь с контрактом: ссылка на конкретные операции в OpenAPI/таблице API.

12 Заключение

Клиент–серверная архитектура остаётся базовым строительным блоком современных ИС. Для её успешной эксплуатации на уровне архитектуры необходимо **связать**: (1) чётко описанный контракт, (2) управляемые качества (надёжность, производительность, безопасность), (3) наблюдаемость и (4) понятную схему развёртывания.